Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Metasploit Frame work 303105324

Mr Bibungsarth Islary
Assistant Professor
CSE (Cyber Security)

Content

1. Client-Side Attack Surface
2. Client-Side Exploitation Methodology
3. Antivirus and Evasion Concepts
4. IDS/IPS Evasion Concepts
5. Human Interface Device Attacks
6. HTA Attack
7. MITM-Based Executable Tampering
8. Trojan Concepts
9. Linux Trojan Analysis
10. PDF Exploitation
11. Microsoft Word Exploitation
12. Android Backdoor Concepts

Client-Side Exploitation

Client-side exploitation targets software, users, or devices that initiate communication with a remote system.

Examples

- Web browsers
- Document readers
- Office applications
- PDF readers
- Email clients
- Operating-system components
- Mobile applications
- USB/HID interfaces

Attacker
↓
Delivery Mechanism
↓
Client
↓
Vulnerable Application
↓
Execution
↓
Session / Impact

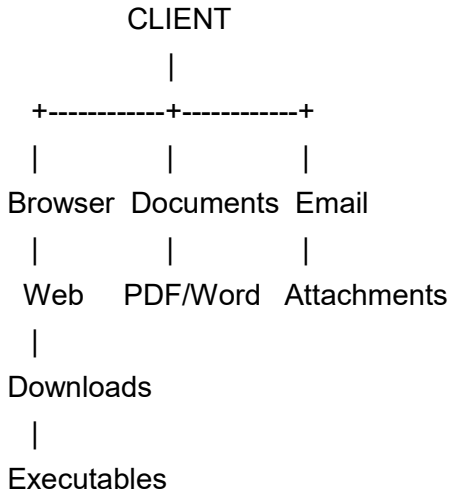
Server-Side vs Client-Side

Server-Side	Client-Side
Targets exposed server	Targets user/client
Often remotely reachable	Often requires user interaction
Service is attack surface	Application/file/device is attack surface
Example: SMB	Example: malicious document
Example: vulnerable HTTP service	Example: vulnerable PDF reader

Server-side exploitation targets the **service**.

Client-side exploitation often targets the **interaction between user, application and content**.

Client-Side Attack Surface



Other attack surfaces:

- USB devices
- HID devices
- Mobile applications
- Removable media
- Network traffic

Client-Side Attack Chain

Reconnaissance

↓

Target Selection

↓

Delivery

↓

User Interaction

↓

Vulnerability / Execution

↓

Payload

↓

Command & Control

↓

Post-Exploitation

Important

Client-side attacks often introduce a **human interaction point** into the attack chain.

DELIVERY MECHANISMS

Client-Side Delivery

Potential delivery mechanisms include:

- Email attachment
- Malicious document
- Compromised website
- Downloaded executable
- USB device
- HID device
- Mobile application
- Modified file

Attacker

↓

Delivery

↓

User / Device

↓

Application

↓

Execution

Antivirus

Antivirus software attempts to identify malicious activity.

Traditional detection may use:

- Signatures
- Hashes
- Known patterns

Modern endpoint protection can also use:

- Behavioral analysis
- Memory inspection
- Process monitoring
- Machine learning
- Reputation
- Cloud-based analysis

How Antivirus Detects Threats

File / Process

↓

Static Analysis

+

Behavioral Analysis

+

Reputation

+

Memory / Runtime Analysis

↓

Detection Decision

Important

Modern security products do not depend only on file hashes.

Antivirus Evasion

AV evasion refers to techniques intended to prevent or reduce detection by security software.

Conceptual categories:

- Obfuscation
- Packing
- Encryption
- Polymorphism
- Runtime behavior changes
- Living-off-the-land techniques

Security lesson

Changing the appearance of malware does not necessarily remove malicious behavior.

Static vs Behavioral Detection

Static Detection	Behavioral Detection
Examines file	Examines activity
Strings	Process behavior
File structure	Network connections
Hash	Child processes
Imports	Memory behavior

Malware
↓
Static Analysis
+
Runtime Behavior
↓
Detection

Why Detection Can Fail

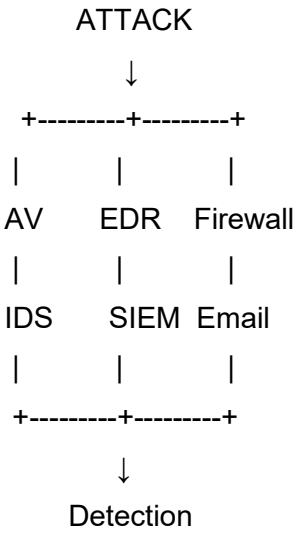
Detection can fail when:

- Threat is previously unknown
- Behavior is ambiguous
- Telemetry is incomplete
- Detection rules are too specific
- Security controls are misconfigured
- Activity resembles legitimate software

But

A sample bypassing one detection layer does not mean it bypasses the entire security stack.

Defense in Depth



Principle

An attacker should not be able to rely on defeating a single security control.

IDS vs IPS

IDS

Intrusion Detection System

- Detects suspicious traffic
- Generates alerts
- Usually does not directly block traffic

IPS

Intrusion Prevention System

- Detects suspicious traffic
- Can block or prevent traffic

Traffic

↓

IDS → Alert

Traffic

↓

IPS → Detect → Block

IDS/IPS Evasion

Conceptual techniques include:

- Traffic fragmentation
- Encoding
- Protocol manipulation
- Traffic obfuscation
- Timing changes
- Alternate communication paths

Defensive principle

Detection should consider both:

Individual packets + complete communication behavior

NETWORK EVASION CONCEPT

Network Evasion

Normal Traffic

↓

IDS/IPS

↓

Detection

Modified Traffic

↓

IDS/IPS

↓

Possible Detection Failure

Modern systems may correlate:

- Source
- Destination
- Timing
- Protocol
- Session behavior
- Endpoint process

Human Interface Device Attacks

HID attacks abuse devices that appear to the operating system as legitimate input devices.

Examples:

- Keyboard emulation
- USB HID devices
- Programmable input devices

HID Device

↓

Operating System

↓

Recognized as Keyboard

↓

Input

HID Attack Concept

The operating system may trust a keyboard-like device without distinguishing whether the input originated from:

- Human
- Physical keyboard
- Emulated keyboard

Attack idea

USB Device

↓

Keyboard Emulation

↓

Rapid Input

↓

Command / Application

HID Attack Chain

Physical Access

↓

Connect HID Device

↓

Device Recognition

↓

Automated Input

↓

Application Interaction

↓

Command Execution

Important

The attack depends heavily on physical access and endpoint configuration.

HID Security Risks

Potential impact:

- Unauthorized commands
- Application launch
- Configuration changes
- Data access
- Malware execution

Defensive controls

- USB device control
- Endpoint policies
- Application allowlisting
- Physical security
- User awareness
- Device monitoring

Detecting HID Attacks

Possible indicators:

- New USB device
- Unexpected device class
- Extremely rapid keyboard input
- Command execution immediately after USB insertion
- Unusual process creation

USB Insertion

+

Rapid Input

+

Command Execution

↓

Suspicious HID Activity

HTA ATTACK

HTA stands for:

HTML Application

HTA files use web technologies but can execute with capabilities beyond a normal browser webpage.

Common extension:

.hta

Concept:

HTML Content

↓

HTA Host

↓

Script / Application Logic

↓

System Interaction

HTA-Based Attack

An attacker may attempt to use an HTA file as an execution mechanism.

User

↓

HTA File

↓

HTA Host

↓

Script Execution

↓

System Interaction

Security concern

HTA combines familiar web content with local execution capabilities.

HTA Attack Chain

Delivery

↓

HTA File

↓

User Execution

↓

HTA Host

↓

Script Execution

↓

Process Creation

↓

Payload / Action

Detection opportunities exist at every stage.

HTA Detection

Monitor:

- HTA file execution
- Script host processes
- Parent-child process relationships
- Unusual command interpreters
- Network connections after HTA execution
- Files downloaded immediately before execution

Important telemetry

HTA

↓

Process

↓

Child Process

↓

Network

MITM & EXECUTABLE TAMPERING

Man-in-the-Middle Attack

A MITM attack places an attacker between two communicating parties.

Client

↓

Attacker

↓

Server

The attacker may attempt to:

- Observe traffic
- Modify traffic
- Redirect traffic
- Inject content

MITM Attack Chain

Client

↓

Traffic Interception

↓

Modification

↓

Forwarded Traffic

↓

Server

Security objective

Maintain confidentiality and integrity of communications.

Executable Tampering

An attacker may attempt to modify software during transmission.

Conceptually:

Original Executable

↓

Network Transfer

↓

Interception

↓

Modification

↓

Modified Executable

↓

Client

Important

Modern secure distribution mechanisms are designed to detect this type of tampering.

WHY HTTPS MATTERS

HTTPS and Integrity

Without secure transport:

Client ←→ Network ←→ Server

↑

Attacker

With authenticated TLS:

Client ←==== TLS ====→ Server

TLS helps provide:

- Confidentiality
- Integrity
- Server authentication

Key idea

Encryption protects more than secrecy. It also helps prevent undetected modification in transit.

Executable Integrity

Important mechanisms:

- Digital signatures
- Hash verification
- Secure distribution
- TLS
- Code signing
- Application allowlisting

Downloaded File

↓

Signature / Hash

↓

Verification

↓

Trusted?

Trojan

A Trojan is software that appears legitimate or useful while performing unauthorized or malicious activity.

User

↓

Legitimate-looking Application

↓

Execution

↓

Hidden Malicious Behavior

Important

Trojan ≠ vulnerability.

A Trojan often relies on **social engineering or user execution**.

Linux Trojan Architecture

Conceptually:

User

↓

Application

↓

+----------------------+

| Legitimate Logic |

| + |

| Hidden Behavior |

+----------------------+

↓

System Activity

Possible malicious behavior:

- Unauthorized communication
- Data collection
- Persistence
- Command execution

Trojan Attack Chain

Delivery

↓

User Trust

↓

Execution

↓

Legitimate Function

+

Hidden Activity

↓

Persistence / Communication

Security lesson

Trust in the application is often the first security boundary being attacked.

Trojan Detection

Analyze:

Static

- Digital signature
- Hash
- Strings
- Imported libraries
- File structure

Dynamic

- Process behavior
- File activity
- Network connections
- Child processes
- Persistence mechanisms

Linux Trojan Analysis

A defensive investigation can ask:

1. What is the file?
2. Who created it?
3. Is it signed?
4. What processes does it create?
5. What files does it access?
6. What network connections does it make?
7. Does it create persistence?
8. What user executes it?

File-Format Based Exploitation

Documents may contain complex structures interpreted by applications.

Examples:

- PDF
- Microsoft Word
- Excel
- PowerPoint
- Image formats
- Archive formats

File

↓

Parser

↓

Application

↓

Vulnerable Component

↓

Execution

WHY FILES CAN BE ATTACK SURFACES

File as an Attack Surface

A document is not simply data.

It may contain:

- Embedded objects
- Scripts
- Macros
- External references
- Complex structures
- Active content

Document

↓

Parser

↓

Application Logic

↓

Potential Vulnerability

PDF Exploitation

PDF readers process complex document structures.

Potential attack areas:

- Parser vulnerabilities
- Embedded content
- JavaScript
- External references
- Memory corruption vulnerabilities

PDF

↓

PDF Reader

↓

Parser

↓

Vulnerability

↓

Potential Code Execution

PDF Attack Chain

Malicious PDF

↓

Delivery

↓

User Opens File

↓

PDF Reader Parses Content

↓

Vulnerability Triggered

↓

Execution

Security dependency

The attack often depends on:

- Vulnerable reader
- Appropriate file structure
- User interaction
- Security controls

PDF Security Analysis

Analyze:

- File metadata
- Embedded objects
- JavaScript
- Suspicious URLs
- File structure
- Digital signatures
- Reader behavior

Defensive workflow

PDF

↓

Static Analysis

↓

Sandbox

↓

Behavior Analysis

↓

Verdict

Word Document Exploitation

Microsoft Word documents can contain active or complex content.

Potential attack surfaces:

- Macros
- Embedded objects
- External references
- Vulnerable parsers
- Malicious document structures

Word File

↓

Microsoft Word

↓

Content Processing

↓

Vulnerable Component

Word-Based Attack Chain

Malicious Document

↓

Email / Download

↓

User Opens Document

↓

Content Processing

↓

Vulnerability / Active Content

↓

Execution

Security controls

- Protected View
- Macro policies
- Application updates
- Email filtering
- EDR

Macro vs Exploit

These concepts should not be confused.

Macro-based attack

Uses document automation functionality.

Exploitation

Takes advantage of a vulnerability in software.

Macro

↓

Abuse of Intended Functionality

Exploit

↓

Abuse of Vulnerable Behavior

Both can result in code execution, but the underlying mechanism differs.

FILE FORMAT DETECTION

Detecting Malicious Documents

Security teams can inspect:

- File origin
- Email metadata
- File hash
- Document structure
- Embedded content
- Child processes
- Network connections
- Application behavior

Strong signal

Document

+

Unexpected Script / Process

+

Network Connection

ANDROID BACKDOOR

Android applications interact with:

- Activities
- Services
- Broadcast receivers
- Content providers
- Permissions
- Network APIs
- Local storage

Android App

↓

Application Components

↓

Permissions

↓

System Resources

Android Backdoor Concept

A backdoor is functionality that provides unauthorized access or control.

Conceptually:

Android Application

↓

Legitimate Functionality

+

Unauthorized Functionality

↓

Remote / Local Control

Important

A backdoor may be hidden inside an otherwise functional application.

ANDROID BACKDOOR CONCEPTUAL ARCHITECTURE

Android Device

|

+--- Legitimate App

|

+--- Hidden Component

|

↓

Communication

|

↓

Remote Endpoint

Possible capabilities:

- Information collection
- Command processing
- Network communication
- Persistence

Android Permissions

Permissions control access to sensitive resources.

Examples:

- Camera
- Microphone
- Location
- Contacts
- Storage
- Network

Security principle

An application requesting excessive permissions should be investigated.

App

↓

Permission Request

↓

Sensitive Resource

Android Backdoor Detection

Analyze:

Static

- APK metadata
- Permissions
- Components
- Strings
- Embedded URLs
- Code structure

Dynamic

- Network traffic
- File access
- Process behavior
- Permission use
- API calls

Android Malware Analysis

APK

↓

Hash

↓

Static Analysis

↓

Permissions

↓

Components

↓

Dynamic Analysis

↓

Network Monitoring

↓

Behavior Analysis

↓

Verdict

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/

